

Deploy a RAG AI Agent in Streamlit using Google ADK and Cloud Run

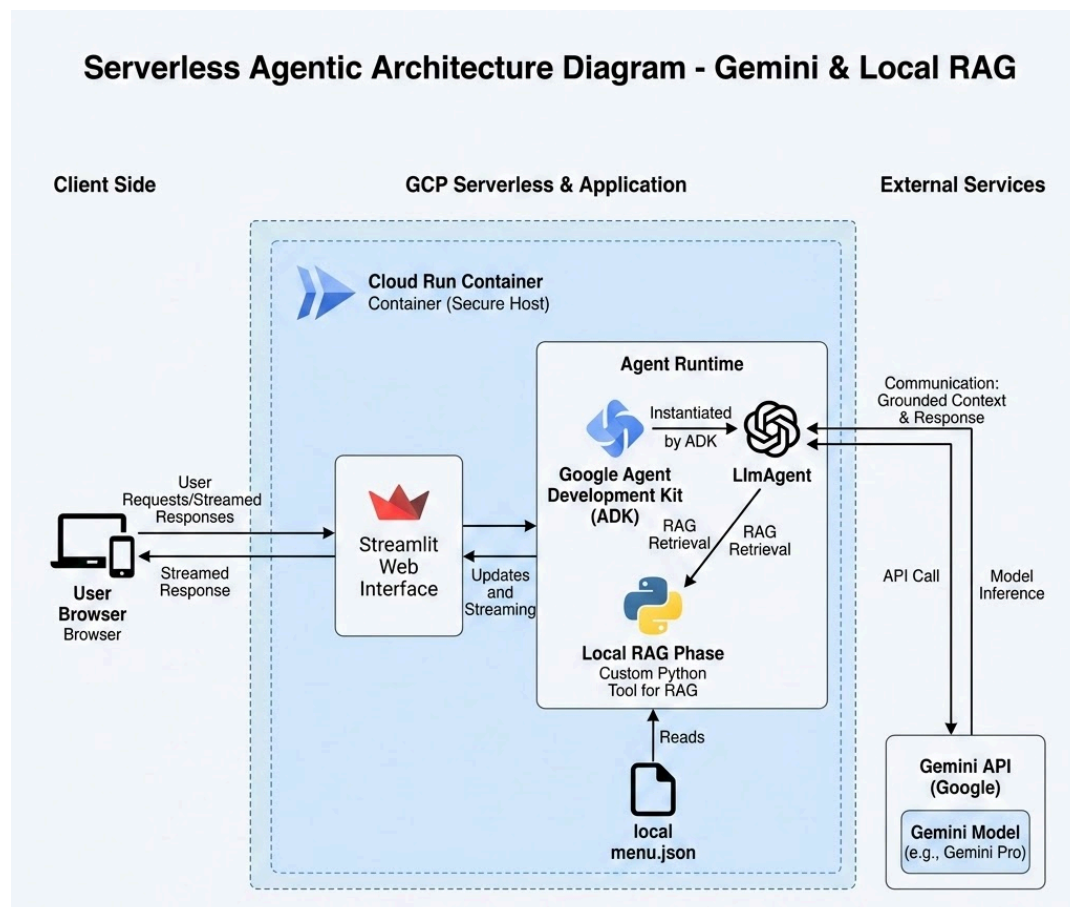
Source link

<https://codelabs.developers.google.com/codelabs/cloud-run/build-streamlit-rag-agent-google-adk-cloud-run#0>

AI Barista agent for a coffee shop. Using Google's open-source Agent Development Kit (ADK) and the Gemini 3.5 Flash model, you'll implement Retrieval-Augmented Generation (RAG) to ground the agent's recommendations in a mock menu dataset. Finally, you'll wrap the agent in a Streamlit user interface and deploy it to Cloud Run.

What you'll do

- Create a RAG data source (`menu.json`) containing coffee items, tags, and allergens.
- Build an AI agent using the ADK `LlmAgent` and connect a Python tool to load the menu data.
- Wrap the agent in a Streamlit chat application that manages conversation history.
- Deploy the Streamlit app to Cloud Run using source-based deployment.
- Test RAG grounding and allergen awareness.



1. Introduction

Data Source: Creating the Mock Menu

Grounding the AI Agent

The foundation of our RAG implementation starts with a local JSON file (menu.json). This acts as the ground truth, preventing the AI Barista from hallucinating non-existent products.

- **Eliminates Complexity:** Bypasses initial database setup overhead to keep workshop focus strictly on core agent construction.
- **Detailed Metadata:** Includes structured item names, prices, custom tags (e.g., 'sweet', 'dairy-free'), and allergens (e.g., 'dairy', 'wheat').
- **Enterprise Transition:** Real-world scenarios swap this local file for a live managed database like Cloud Firestore or AlloyDB to dynamic update menus.

```
// menu.json (Ground Truth Data)
[
  {
    "name": "Pumpkin Latte",
    "description": "Seasonal espresso drink",
    "price": 4.75,
    "tags": ["sweet", "seasonal"],
    "allergens": ["dairy"]
  },
  {
    "name": "Croissant",
    "description": "Flaky bakery pastry",
    "price": 3.50,
    "tags": ["bakery", "sweet"],
    "allergens": ["wheat", "dairy"]
  }
]
```

Define Custom Tools

Python function `get_menu()`

```
# Loads menu data as context
def get_menu():
    menu_data = load_json('menu.json')
    return str(menu_data)
```

- **Core Intelligence:** Defined in the `agent.py` file to specify custom capabilities.
- **Dynamic Context:** Loads `menu.json` dynamically and returns it as a string to serve as our retrieval tool.

Configure 'LlmAgent'

Initialize and bind `barista_agent`

```
# Configure and bind custom tool
barista_agent = LlmAgent(
    name="barista_agent",
    model="gemini-3.5-flash",
    tools=[get_menu],
    system_instruction=barista_prompt
)
```

- **Optimized Model:** Powered by `gemini-3.5-flash` for fast, cost-efficient, interactive chat.
- **Persona Bounds:** System prompt restricts agent to recommend items **ONLY** from the menu.
- **Token Efficiency:** Function is bound to the agent's tools to fetch context only when required.

Creating the Streamlit Chat Interface (app.py)

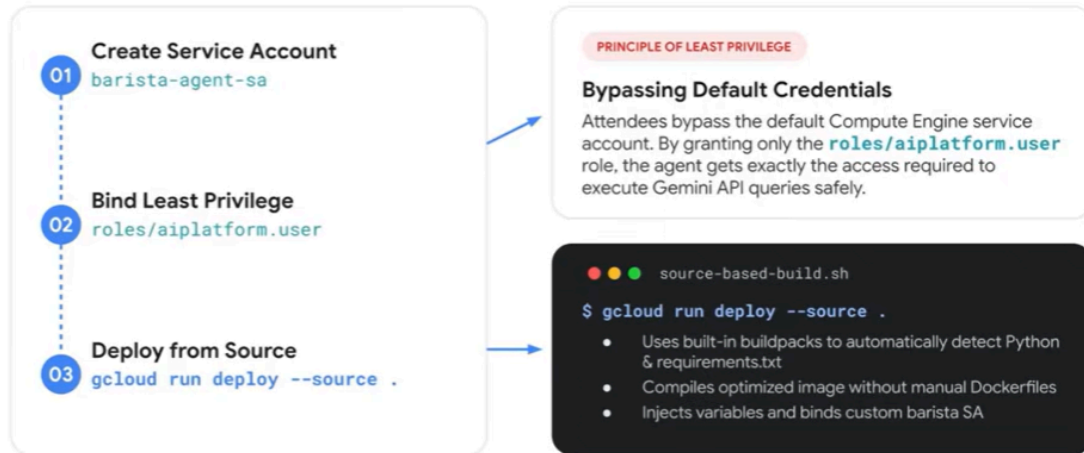
Memory in Stateless Frameworks

Because Streamlit is stateless, we use `st.session_state` to persist session context across active browser connections.

- **State Management:** Maintains conversation history and generates a unique ID per active browser connection.
- **Dynamic Sidebar:** Reads raw `menu.json` data and renders the current cafe offerings instantly.
- **Volatile Storage:** App state lives in-memory; refreshing the tab will reset your current conversation context.

```
// menu.json (Ground Truth Data)
[
  {
    "name": "Pumpkin Latte",
    "description": "Seasonal espresso drink",
    "price": 4.75,
    "tags": ["sweet", "seasonal"],
    "allergens": ["dairy"]
  },
  {
    "name": "Croissant",
    "description": "Flaky bakery pastry",
    "price": 3.50,
    "tags": ["bakery", "sweet"],
    "allergens": ["wheat", "dairy"]
  }
]
```

Cloud Run Deployment



Verification: Testing Grounded RAG Behavior

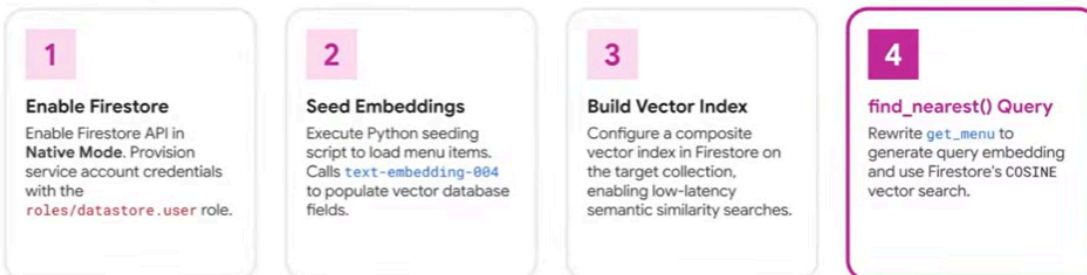


Going Production-Ready

Grounding Agent in Firestore with Vector Search

Migrate from a local `menu.json` file to a production-ready, scalable Cloud Firestore database. This ensures dynamic menu updates can be served instantly to the LLM agent without requiring container redeployments.

The rewritten tool dynamically converts search queries to vector embeddings using `text-embedding-004`, executing nearest-neighbor search directly on Firestore.



2. Before you begin

```
Welcome to Cloud Shell! Type "help" to get started.
Your Cloud Platform project in this session is set to seraphic-lock-480916-i3.
Use `gcloud config set project [PROJECT_ID]` to change to a different project.
lipsasahoo1805@cloudshell:~ (seraphic-lock-480916-i3)$ gcloud config get project
Your active configuration is: [cloudshell-29209]
seraphic-lock-480916-i3
lipsasahoo1805@cloudshell:~ (seraphic-lock-480916-i3)$ gcloud services enable \
run.googleapis.com \
aiplatform.googleapis.com \
cloudbuild.googleapis.com
Operation "operations/acf.p2-465708172848-923fa9f0-4a8b-4740-be61-b861aff60a71" finished successfully.
lipsasahoo1805@cloudshell:~ (seraphic-lock-480916-i3)$ gcloud services list --enabled
NAME: aiplatform.googleapis.com
TITLE: Agent Platform API

NAME: analyticshub.googleapis.com
TITLE: Analytics Hub API

NAME: artifactregistry.googleapis.com
TITLE: Artifact Registry API

NAME: bigquery.googleapis.com
TITLE: BigQuery API

NAME: bigqueryconnection.googleapis.com
TITLE: BigQuery Connection API

NAME: bigquerydatapolicy.googleapis.com
TITLE: BigQuery Data Policy API

NAME: bigquerydatatransfer.googleapis.com
TITLE: BigQuery Data Transfer API

NAME: bigquerymigration.googleapis.com
TITLE: BigQuery Migration API

NAME: bigqueryreservation.googleapis.com
TITLE: BigQuery Reservation API
```

3. Set up your project

```
lipsasahoo1805@cloudshell:~ (seraphic-lock-480916-i3) $ export PROJECT_ID=$(gcloud config get-value project)
Your active configuration is: [cloudshell-29209]
lipsasahoo1805@cloudshell:~ (seraphic-lock-480916-i3) $ export REGION="asia-south1-a"
lipsasahoo1805@cloudshell:~ (seraphic-lock-480916-i3) $ echo $REGION
asia-south1-a
lipsasahoo1805@cloudshell:~ (seraphic-lock-480916-i3) $ echo $PROJECT_ID
seraphic-lock-480916-i3
lipsasahoo1805@cloudshell:~ (seraphic-lock-480916-i3) $ ls
README-cloudshell.txt
lipsasahoo1805@cloudshell:~ (seraphic-lock-480916-i3) $ mkdir coffee-barista-agent
lipsasahoo1805@cloudshell:~ (seraphic-lock-480916-i3) $ ls
coffee-barista-agent README-cloudshell.txt
lipsasahoo1805@cloudshell:~ (seraphic-lock-480916-i3) $ cd coffee-barista-agent/
lipsasahoo1805@cloudshell:~/coffee-barista-agent (seraphic-lock-480916-i3) $
```

4. Create the mock menu data source

The screenshot shows the Google Cloud Cloud Shell Editor interface. The Explorer pane on the left displays the directory structure: LIPSASAHOO1805 > coffee-barista-agent > menu.json. The main editor pane shows the content of menu.json, which is a JSON array of four coffee items. The status bar at the bottom indicates the file is menu.json, line 58, column 2.

```
1 {
2   {
3     "name": "Espresso Solo",
4     "description": "A single shot of rich, bold espresso.",
5     "price": 2.50,
6     "tags": ["strong", "hot", "dairy-free", "sugar-free"],
7     "allergens": []
8   },
9   {
10    "name": "Oat Milk Honey Latte",
11    "description": "Creamy steamed oat milk with espresso and a touch of honey.",
12    "price": 5.00,
13    "tags": ["sweet", "hot", "dairy-free"],
14    "allergens": []
15  },
16  {
17    "name": "Cold Brew Coffee",
18    "description": "Smooth, slow-steeped cold brew served over ice.",
19    "price": 4.00,
20    "tags": ["strong", "cold", "dairy-free", "sugar-free"],
21    "allergens": []
22  },
23  {
24    "name": "Seasonal Pumpkin Latte",
25    "description": "Spiced pumpkin sauce, espresso, and steamed milk, topped with whipped cream."
26  }
27 }
```

```
lipsasahoo1805@cloudshell:~/coffee-barista-agent (seraphic-lock-480916-i3) $ cd coffee-barista-agent/
lipsasahoo1805@cloudshell:~/coffee-barista-agent (seraphic-lock-480916-i3) $ cloudshell edit menu.json
lipsasahoo1805@cloudshell:~/coffee-barista-agent (seraphic-lock-480916-i3) $

lipsasahoo1805@cloudshell:~/coffee-barista-agent (seraphic-lock-480916-i3) $ cat menu.json | python3 -m json.tool > /dev/null && echo "Valid JSON!"
Valid JSON!805@cloudshell:~/coffee-barista-agent (seraphic-lock-480916-i3) $

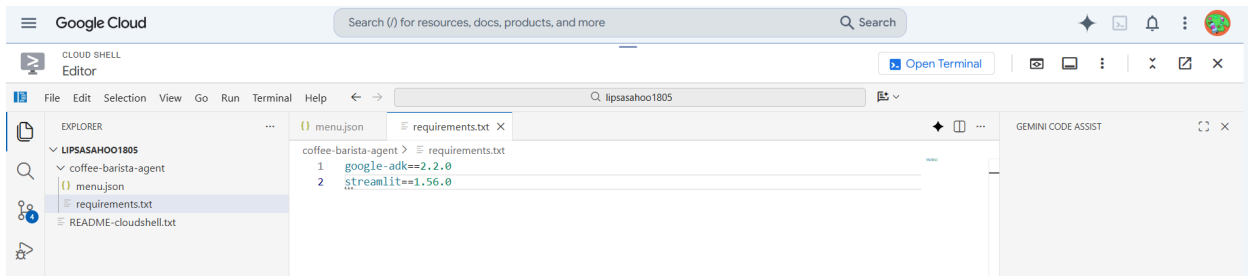
lipsasahoo1805@cloudshell:~/coffee-barista-agent (seraphic-lock-480916-i3) $ cat menu.json | python3 -m json.tool > /dev/null && echo "Valid JSON!"
Valid JSON!805@cloudshell:~/coffee-barista-agent (seraphic-lock-480916-i3) $

lipsasahoo1805@cloudshell:~/coffee-barista-agent (seraphic-lock-480916-i3) $ cat menu.json | python3 -m json.tool > /dev/null && echo "Valid JSON!"
Valid JSON!805@cloudshell:~/coffee-barista-agent (seraphic-lock-480916-i3) $

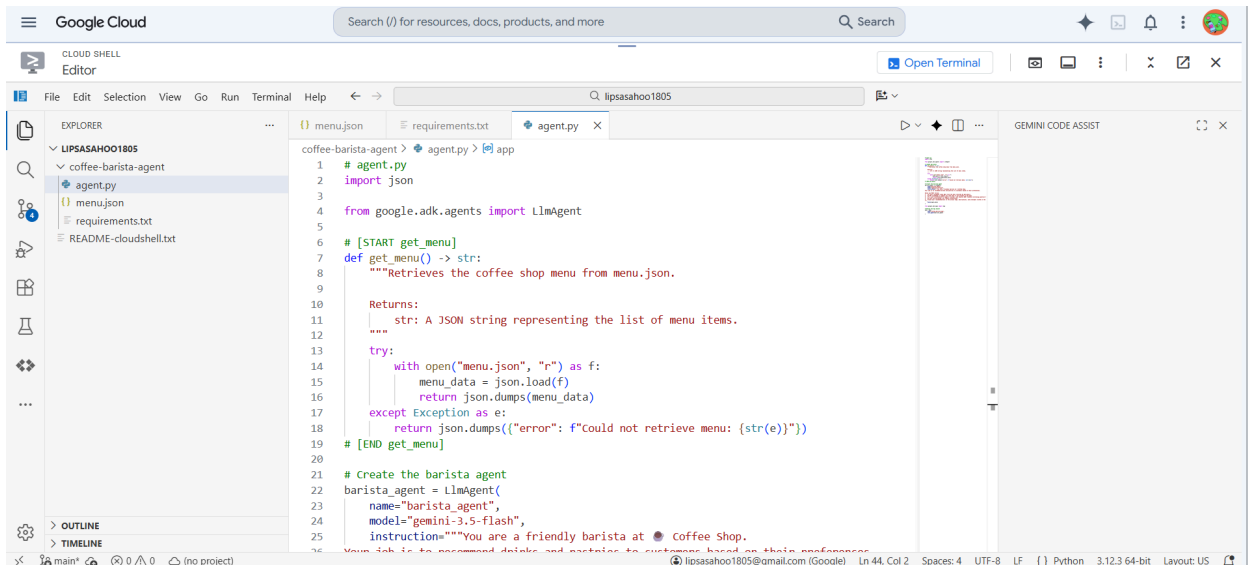
lipsasahoo1805@cloudshell:~/coffee-barista-agent (seraphic-lock-480916-i3) $ cat menu.json | python3 -m json.tool > /dev/null && echo "Valid JSON!"
Valid JSON!805@cloudshell:~/coffee-barista-agent (seraphic-lock-480916-i3) $
```

5. Build the ADK agent

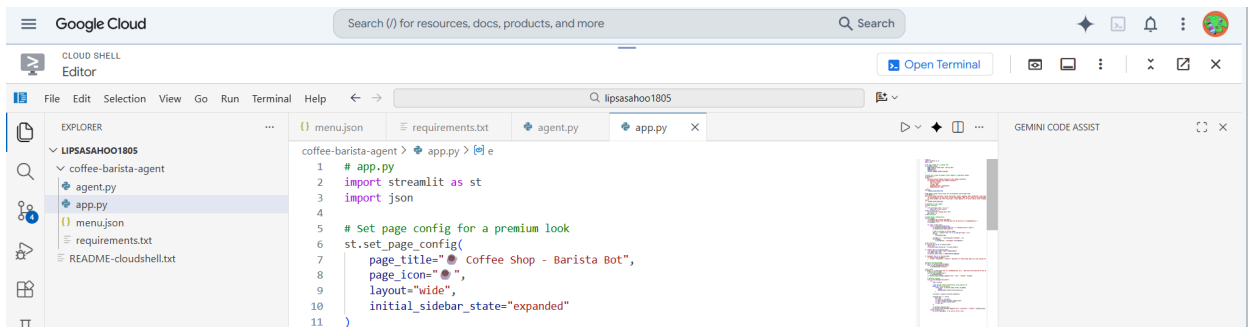
```
lipsasahoo1805@cloudshell:~/coffee-barista-agent (seraphic-lock-480916-i3)$
lipsasahoo1805@cloudshell:~/coffee-barista-agent (seraphic-lock-480916-i3)$
lipsasahoo1805@cloudshell:~/coffee-barista-agent (seraphic-lock-480916-i3)$ cloudshell edit requirements.txt
lipsasahoo1805@cloudshell:~/coffee-barista-agent (seraphic-lock-480916-i3)$
lipsasahoo1805@cloudshell:~/coffee-barista-agent (seraphic-lock-480916-i3)$ cloudshell edit agent.py
lipsasahoo1805@cloudshell:~/coffee-barista-agent (seraphic-lock-480916-i3)$
lipsasahoo1805@cloudshell:~/coffee-barista-agent (seraphic-lock-480916-i3)$
lipsasahoo1805@cloudshell:~/coffee-barista-agent (seraphic-lock-480916-i3)$ cloudshell edit app.py
lipsasahoo1805@cloudshell:~/coffee-barista-agent (seraphic-lock-480916-i3)$
```



agent.py - agent setup



app.py - frontend of the application - using streamlit web application



6. Deploy the agent to Cloud Run

```
lipsasahoo1805@cloudshell:~/coffee-barista-agent (seraphic-lock-480916-i3)$ gcloud iam service-accounts create barista-agent-sa \
--description="Service account for Coffee Barista ADK agent on Cloud Run" \
--display-name="Barista Agent Service Account"
Created service account [barista-agent-sa].
Service account email: barista-agent-sa@seraphic-lock-480916-i3.iam.gserviceaccount.com
lipsasahoo1805@cloudshell:~/coffee-barista-agent (seraphic-lock-480916-i3)$ gcloud projects add-iam-policy-binding $PROJECT_ID \
--member="serviceAccount:barista-agent-sa@$PROJECT_ID.iam.gserviceaccount.com" \
--role="roles/aiplatform.user"
Updated IAM policy for project [seraphic-lock-480916-i3].
bindings:
- members:
  - serviceAccount:barista-agent-sa@seraphic-lock-480916-i3.iam.gserviceaccount.com
  role: roles/aiplatform.user
- members:
  - serviceAccount:465708172848@cloudbuild.gserviceaccount.com
  role: roles/cloudbuild.builds.builder
- members:
  - serviceAccount:service-465708172848@gcp-sa-cloudbuild.iam.gserviceaccount.com
  role: roles/cloudbuild.serviceAgent
- members:
  - serviceAccount:service-465708172848@containerregistry.iam.gserviceaccount.com
  role: roles/containerregistry.ServiceAgent
- members:
  - serviceAccount:465708172848-compute@developer.gserviceaccount.com
  role: roles/editor
- members:
  - user:lipsasahoo1805@gmail.com
  role: roles/owner
- members:
  - serviceAccount:service-465708172848@gcp-sa-pubsub.iam.gserviceaccount.com
  role: roles/pubsub.serviceAgent
- members:
  - serviceAccount:service-465708172848@serverless-robot-prod.iam.gserviceaccount.com
  role: roles/run.serviceAgent
etag: BwZYcuJYVrw=
version: 1
lipsasahoo1805@cloudshell:~/coffee-barista-agent (seraphic-lock-480916-i3)$
```

```
lipsasahoo1805@cloudshell:~/coffee-barista-agent (seraphic-lock-480916-i3)$ gcloud run deploy coffee-barista \
--source . \
--region asia-south1 \
--allow-unauthenticated \
--labels dev-tutorial=codelab-streamlit-rag-adk \
--command "/cnb/lifecycle/launcher" \
--args "sh,-c,python3 -m streamlit run app.py --server.port=$PORT --server.address=0.0.0.0 --server.enableCORS=false --server.enableXsrfProtection=false" \
--service-account "barista-agent-sa@$PROJECT_ID.iam.gserviceaccount.com" \
--set-env-vars GOOGLE_CLOUD_PROJECT=$PROJECT_ID,GOOGLE_CLOUD_LOCATION=global \
--image asia-south1-docker.pkg.dev/$PROJECT_ID/coffee-barista-repo/coffee-barista:latest
Building using Buildpacks and deploying container to Cloud Run service [coffee-barista] in project [seraphic-lock-480916-i3] region [asia-south1]
Building and deploying new service...
Validating configuration...done
Uploading sources...done
Building Container... Logs are available at [ https://console.cloud.google.com/cloud-build/builds;region=asia-south1/1f99508b-713a-4fa8-b93d-b76ef841e7bf?project=465708172848 ]....done
Setting IAM Policy...done
Creating Revision...done
Routing traffic...done
Done.
Service [coffee-barista] revision [coffee-barista-00001-x8z] has been deployed and is serving 100 percent of traffic.
Service URL: https://coffee-barista-465708172848.asia-south1.run.app
lipsasahoo1805@cloudshell:~/coffee-barista-agent (seraphic-lock-480916-i3)$
```

<https://coffee-barista-465708172848.asia-south1.run.app/>

7. Test the RAG behavior

8. Optional: Ground your agent in Firestore using Vector Search

```
lipsasahool805@cloudshell:~/coffee-barista-agent (seraphic-lock-480916-i3)$ gcloud services enable firestore.googleapis.com

gcloud firestore databases create --database="coffee-menu" --location=$REGION
Operation "operations/acat.p2-465708172848-2811f6bc-a17d-4699-9099-faa230fb13da" finished successfully.
done: true
metadata:
  '@type': type.googleapis.com/google.firestore.admin.v1.CreateDatabaseMetadata
name: projects/seraphic-lock-480916-i3/databases/coffee-menu/operations/qU99msKMJYwCSjW115_SrxAgMWh0dW9zLWFpc2ELIgUQAzeJu6gQBtFXnPYIDAovGg
response:
  '@type': type.googleapis.com/google.firestore.admin.v1.Database
  appEngineIntegrationMode: DISABLED
  concurrencyMode: PESSIMISTIC
  createTime: '2026-08-07T12:24:22.316825Z'
  databaseEdition: STANDARD
  deleteProtectionState: DELETE_PROTECTION_DISABLED
  earliestVersionTime: '2026-08-07T12:24:22.316825Z'
  etag: ILXt86zBjpYDMJm+86zBjpYD
  freeTier: true
  locationId: asia-south1
  name: projects/seraphic-lock-480916-i3/databases/coffee-menu
  pointInTimeRecoveryEnablement: POINT_IN_TIME_RECOVERY_DISABLED
  realtimeUpdatesMode: REALTIME_UPDATES_MODE_ENABLED
  type: FIRESTORE_NATIVE
  uid: afd29fd7-b535-4a9c-8525-8cc29a7d4fa9
  updateTime: '2026-08-07T12:24:22.316825Z'
  versionRetentionPeriod: 3600s
lipsasahool805@cloudshell:~/coffee-barista-agent (seraphic-lock-480916-i3)$
```

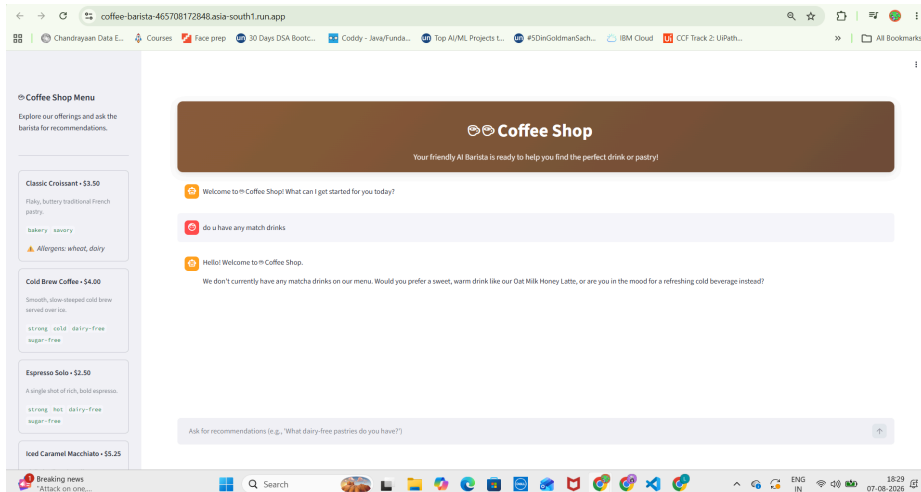
```
lipsasahool805@cloudshell:~/coffee-barista-agent (seraphic-lock-480916-i3)$ python3 seed.py
Firestore menu collection seeded with vector embeddings successfully!
lipsasahool805@cloudshell:~/coffee-barista-agent (seraphic-lock-480916-i3)$
```

```
lipsasahool805@cloudshell:~/coffee-barista-agent (seraphic-lock-480916-i3)$ gcloud firestore indexes composite create \
--collection-group=menu \
--query-scope=COLLECTION \
--database="coffee-menu" \
--field-config=field-path=embedding,vector-config='{"dimension": "768", "flat": "{}"}'
Create request issued
Waiting for operation [projects/seraphic-lock-480916-i3/databases/coffee-menu/operations/S00UaPhqT2BQ0LD0CoDIDIZMjJhMjFmNzgzZC0wYjI4L1l1NTQzMTU3Yy00MmVjMTRjYyQac2VuaWxlcGlwC0pBEg] to complete...done.
Created index [C1CAgDjXh4EK].
```

The screenshot shows the Google Cloud Firestore console. On the left, the 'Database' sidebar is open, showing the 'coffee-menu' database. The 'menu' collection is expanded, displaying a list of documents. The 'cold-brew-coffee' document is selected, showing its fields: 'allergens', 'description', 'embedding', 'name', 'price', and 'tags'. The 'embedding' field is highlighted, showing a vector of 768 dimensions. The 'tags' field is also visible, showing a list of tags: 'strong', 'cold', 'dairy-free', and 'sugar-free'.

```
lipsasahool805@cloudshell:~/coffee-barista-agent (seraphic-lock-480916-i3)$ gcloud projects add-iam-policy-binding $PROJECT_ID \
--member="serviceAccount:barista-agent-sa@$PROJECT_ID.iam.gserviceaccount.com" \
--role="roles/datastore.user"
Updated IAM policy for project [seraphic-lock-480916-i3].
bindings:
- members:
  - serviceAccount:barista-agent-sa@seraphic-lock-480916-i3.iam.gserviceaccount.com
  role: roles/aiplatform.user
- members:
  - serviceAccount:service-465708172848@gcp-sa-artifactregistry.iam.gserviceaccount.com
  role: roles/artifactregistry.serviceAgent
- members:
  - serviceAccount:465708172848@cloudbuild.gserviceaccount.com
  role: roles/cloudbuild.builds.builder
- members:
  - serviceAccount:service-465708172848@gcp-sa-cloudbuild.iam.gserviceaccount.com
  role: roles/cloudbuild.serviceAgent
- members:
  - serviceAccount:service-465708172848@containerregistry.iam.gserviceaccount.com
  role: roles/containerregistry.serviceAgent
- members:
  - serviceAccount:barista-agent-sa@seraphic-lock-480916-i3.iam.gserviceaccount.com
  role: roles/datastore.user
- members:
  - serviceAccount:465708172848-compute@developer.gserviceaccount.com
  role: roles/editor
```

Now Redeploy after do the changes in the [agent.py](#) and [app.py](#) for the production ready by using firestore database , where the data is stored as embeddings, and use vector search



```
lipsasahool1805@cloudshell:~/coffee-barista-agent (seraphic-lock-480916-13)$ python3 -c "
import os
from google import genai
from google.cloud import firestore
from google.cloud.firestore_v1.vector import Vector

db = firestore.Client(database='coffee-menu')
client = genai.Client(
    vertexai=True,
    project=os.environ.get('PROJECT_ID'),
    location=os.environ.get('REGION', 'us-central1')
)

name = 'Matcha Green Tea Latte'
desc = 'Creamy steamed oat milk infused with premium Japanese matcha powder.'
res = client.models.embed_content(
    model='text-embedding-004',
    contents=f'{name}: {desc}'
)
embedding = res.embeddings[0].values

db.collection('menu').document('matcha-latte').set({
    'name': name,
    'description': desc,
})
print('Successfully added Matcha Latte with vector embeddings!')
Successfully added Matcha Latte with vector embeddings!
lipsasahool1805@cloudshell:~/coffee-barista-agent (seraphic-lock-480916-13)$ "
```

